

UNIVERSITY COLLEGE LONDON

EXAMINATION FOR INTERNAL STUDENTS

MODULE CODE : **COMP0178**

ASSESSMENT PATTERN: **COMP0178A7PG 001**

MODULE NAME : **COMP0178 - Database Fundamentals**

LEVEL : **Postgraduate**

DATE: : **06-May-2025**

TIME : **10:00**

DURATION : **2 Hours**

This paper is suitable for candidates who attended classes for this module in the following academic year(s):

**Year
2024-25**

EXAMINATION PAPER CANNOT BE REMOVED FROM THE EXAM HALL. PLACE EXAM PAPER AND ALL COMPLETED SCRIPTS INSIDE THE EXAMINATION ENVELOPE

Hall instructions	N/A
Additional materials	None
Standard Calculators	Yes
Non-Standard Calculators	No

TURN OVER

Suitable for Cohort(s): 2024/25, 2023/24, 2022/23, 2021/22

There are FIVE questions in total. Answer ALL questions. Standard calculators are permitted.

A maximum of 100 marks is available. The marks available for each part of each question are indicated in square brackets [n].

1) [Database design]

Given the specification below, create an Entity Relationship Diagram to capture the data needed to support the system. Include important entities and relationships. Attributes may be omitted from the diagram unless necessary to explain a design. Relationship multiplicities must be included.

The system you are creating is a restaurant review app. The site contains information about various local restaurants, such as the name of each restaurant, its address, geographical location, and other various features that people might want to filter for (price, type of cuisine, etc.). Users make accounts with the site and can create reviews on restaurants that includes both numerical rating and a text review. Users can also make public or private lists of restaurants (e.g. for recommendations, marking restaurants they want to try, etc.). Each list has a title set by the user, a unique identifier, and a list of restaurants.

[Total for Question 1: 20 marks]

2) [Query processing]

Consider the following database schema used to support the operations of an online store:

Customers (custID, firstname, familyname, town, state)

Items (itemID, description, unitcost, stocklevel)

LineItems (orderID, itemID, quantity, despatched)

Orders (orderID, custID, date)

[Question 2 continues over page]

[Question 2 continued]

- (a) Write an SQL query to answer a question that the store might have. An example of such a question would be the total quantity of item #323 that was bought by customers, grouped by customer state. You must use a different question and say what it is. Your query should require at least three tables.

[5 marks]

- (b) Translate the SQL query you wrote in the previous part to a relational algebra expression that accomplishes the same task.

[5 marks]

- (c) Translate the relational algebra expression you gave above into (1) a relational algebra tree that matches exactly the expression, and (2) a second relational algebra tree that is different from (not identical to) but equivalent in meaning to the first.

[10 marks]

- (d) For the relational algebra trees you've given, estimate the number of disk accesses that would be required under the following assumptions:
- No indexes exist.
 - Tuples are read one at a time (1 tuple = 1 disk read).
 - All tables fit entirely in memory.
 - All intermediate calculations are stored to disk. However, the final result is returned directly to the user and not stored.
 - The Customers table contains 3,000 rows, Items table contains 600 rows, LineItems table contains 10,000 rows and the Orders tables contains 6,000 rows.

[Question 2 continues over page]

[Question 2 continued]

If there is any unknown information needed to make your calculation (for example, a statistic like the number of rows resulting from a selection must be invented in order to complete a calculation), clearly state the assumptions you make.

[10 marks]

[Total for Question 2: 30 marks]

3) [Transactions]

(a) Locking:

T1: Lock_write(y) Read(y) Lock_write(x) Read(x) Write(y) Write(x) Unlock(y) Unlock(x)	T2: Lock_write(x) Lock_write(y) Read(x) Unlock(x) Read(y) Write(y) Unlock(y)
---	---

Table 1 Two transactions with locking

- (i) Is an inconsistent schedule possible with the above transactions? Briefly explain why or why not (1-2 sentences). (5 marks)
- (ii) Is deadlock possible with the above transactions? Briefly explain why or why not (1-2 sentences). (5 marks)

[10 marks]

- (b) Give an example of an interleaved schedule (involving two or more transactions) that is **NOT** recoverable.

[10 marks]

[Question 3 continues over page]

[Question 3 continued]

- (c) Is the following interleaving conflict serializable or not? Explain your answer. In addition, if it is conflict serializable, what is the equivalent serial schedule?

T1: Read(A)
T2: Read(B)
T3: Read(A)
T3: Write(B)
T2: Write(A)

Table 2 Conflict serializability interleaving

[5 marks]

[Total for Question 3: 25 marks]

4) [Distributed databases]

Consider a distributed database supporting a social media system where users make posts. There are four main tables:

User (userId, email, name, password, etc.)

UserPreferences (userId, emailNotifications, siteDisplay, etc.)

UserFriends (userId, friendId, dateFriended, etc.)

UserPosts (postId, author {FK to User}, date, contents, etc.)

There are four sites (S1-S4), each storing roughly one-fourth of users' data, fragmented based on userID.

- (a) Say that you want to pull up all the info related to the user with userId 788251: basic user info, preferences, and recent posts. How would this query be carried out in this distributed database? Give a brief summary (1-2 sentences).

[5 marks]

- (b) Say that you are calculating some statistics on site activity and want to calculate the average number of posts per user on the site during a specific year (2024). Describe two different strategies that could be used to carry out this distributed query, and which one would be more efficient, if there is any difference.

[10 marks]

[Total for Question 4: 15 marks]

5) [Non-relational databases/general]

- (a) Given an XML document with the following structure, write a FLWOR or XPath-style query to get a list of properties that have not been visited. (Exactly correct syntax is not needed.)

```
<PROPERTIES>
  <PROPERTY>
    <ADDRESS>
      <NUMBER>123</NUMBER>
      <STREET>Main Street</STREET>
      <CITY>London</CITY>
      <MONTHLYRENT>1000</MONTHLYRENT>
      <TIMESVISITED>12</TIMESVISITED>
    </PROPERTY>
  <PROPERTY>
    ...
  </PROPERTY>
  ...
</PROPERTIES>
```

[5 marks]

- (b) Can relational databases and semi-structured databases be used together?
Briefly explain your answer (1-2 sentences).

[5 marks]

[Total for Question 5: 10 marks]

END OF PAPER